Lift some arbitrary CTAD restrictions

Document #:	P4175R0 [Latest] [Status]
Date:	2026-04-04
Project:	Programming Language C++
Audience:	Evolution Working Group Core Working Group
Reply-to:	Matheus Izvekov <mizvekov@gmail.com>

Contents

§ 1 Introduction

This paper proposes to relax some Class Template Argument deduction (CTAD) restrictions which were originally put in place for no well motivated reason other than simplicity and being conservative.

These are already accepted in most implementations, and would be an easy change for the others.

§ 2 Deduction guides can only be declared to produce a simple-template-id.

When declaring a deduction guide, currently it's "trailing return type" is restricted to the exact spelling of a template specialization of the template-name associated to it (which precedes the parameter declarations).

So something like this is supposed to work:

```
template <class T> struct Templ { };
Templ(int) -> Templ<int>;
```

But not this:

```
using TemplInt = Templ<int>;
Templ(int) -> TemplInt;
```

The last example doesn't spell the template specialization directly, so it's supposed to be rejected, but in practice only Clang rejects it.

Giving names to certain template specializations is common practice; think of cases like `std::string`, where spelling `std::basic_string<char>` feels unnatural.

According to the author of the paper which added the CTAD feature, this restriction was put in place for simplicity of specification and because a use case was not put forward at the time.

This paper proposes to lift that restriction, and accept any type which is a non-cv-qualified template specialization of the associated template, no matter how it's spelled in source code.

§ 3 Restriction on dependent nested names within the Alias Template CTAD feature

The CTAD feature came with some restrictions on dependent stuffs.

For example, a type template parameter couldn't be used as a deducible template. But as shown in [CWG3003], this was relied on by the standard library, and all implementations allowed this anyway.

The fix for [CWG3003], which was accepted in Croydon 2026, was the minimal fix concerned with the standard library use, and was rushed as so in order to avoid shipping this defect.

But it left in place another arbitrary restriction, which also currently all implementations already allow: the nested-name-specifier of the defining-type-id for the alias template in the Alias CTAD feature can't be dependent.

Example:

```
template <class T> struct A {
  void f() {
    typename T::XX x(42);
  }
};
struct B {
```

```
  template <class T> struct XX { XX(T); };
};
A<B> x;
```

All implementations currently accept this.

In general, for most features in C++, we accept template definitions which have valid instantiations (even if some other instantiations may turn out to be invalid), so we avoid outright rejecting dependent constructs, unless in a case-by-case basis we know they can have no valid instantiations.

This happens here in this last example: if we manually substitute the template definitions in our heads, the outcome is a use of CTAD which is unambiguously valid.

This is not a contrived example, and naturally users would have no reason to believe this is not supposed to work, unless they are language lawyers.

Since all implementations already accept this, it's highly likely enough time has passed where users could have started depending on this, similar to the standard library motivating use for [CWG3003].

§ 4 Wording

Modify 9.2.9.3 [dcl.type.simple] paragraph 3:

> 3 A placeholder-type-specifier is a placeholder for a type to be deduced ([dcl.spec.auto]). A type-specifier is a placeholder for a deduced class type ([dcl.type.class.deduct]) if either - it is of the form typename$_{opt}$ nested-name-specifier$_{opt}$ template-name or - it is of the form typename$_{opt}$ splice-specifier and the splice-specifier designates a class template or alias template. ~~The nested-name-specifier or splice-specifier, if any, shall be non-dependent and~~ the template-name or splice-specifier shall designate a deducible template. A deducible template is either a class template or is an alias template whose defining-type-id is of the form …

Modify 13.4.4 [temp.arg.template] paragraph 1 and 3:

> 1 Deduction guides are used when a template-name or splice-type-specifier appears as a type specifier for a deduced class type ([dcl.type.class.deduct]). Deduction guides are not found by name lookup. Instead, when performing class template argument deduction ([over.match.class.deduct]), all reachable deduction guides declared for the class template are considered. deduction-guide: explicit-specifier$_{opt}$ template-name (parameter-declaration-clause) -> ~~simple-template-id~~simple-type-specifier requires-clause$_{opt}$;
>
> 2 [*Example: […] — end example*]

[3] The same restrictions apply to the parameter-declaration-clause of a deduction guide as in a function declaration ([dcl.fct]), except that a generic parameter type placeholder ([dcl.spec.auto]) shall not appear in the parameter-declaration-clause of a deduction guide. The ~~simple-template-id~~template-name shall name a class template ~~specialization~~. ~~The template-name shall be the same identifier as the template-name of the simple-template-id~~The simple-type-specifier shall be equivalent to a specialization of the class template named by the template-name. A deduction-guide shall inhabit the scope to which the corresponding class template belongs and, for a member class template, have the same access. Two deduction guide declarations for the same class template shall not have equivalent parameter-declaration-clauses if either is reachable from the other.

§ 5 Acknowledgements

Thanks to Richard Smith, as the author of the CTAD feature, for explaining the reasoning behind the restrictions and also reviewing this document.

§ 6 References

[CWG3003] Hubert Tong. 2025-02-02. Naming a deducible template for class template argument deduction.
https://wg21.link/cwg3003